SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 3 – COMPLETE SOLUTIONS

CO3: Examine cryptographic hash algorithms and digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication and non-repudiation.

Course: Cryptography and Network Security (CSA51)

Document Type: Comprehensive Model Solutions (Student A + Student B + Joint)

Scope: Q1 – Q7 Full Coverage with Theory, Derivation, Experiment Guidance & Design

This document provides rigorous, examination-ready solutions. Experimental sections include clear design of inputs, measurement methodology, expected mathematical results, and interpretation guidelines. All joint design questions are answered with justified recommendations.

Q1 – What Makes a Hash Function Trustworthy? (6 Marks)

Part A – Avalanche and Fixed-Length Digests

Student A – Design & Mathematical Analysis

1. Properties as Attack Scenarios

Pre-image resistance (one-wayness): Given only a digest h, it is computationally infeasible to find any message m such that H(m) = h. Attack scenario: An attacker who obtains a password hash cannot recover the original password by inversion. Security strength for an n-bit ideal hash is approximately 2^n work.

Second pre-image resistance: Given a specific message m_1, it is infeasible to find a different $m_2 \neq m_1$ such that $H(m_1) = H(m_2)$. Attack scenario: An attacker who has a signed legitimate document cannot produce a different fraudulent document that produces the identical digest (and therefore the same signature remains valid). Security strength $\approx 2^n$.

Collision resistance: It is infeasible to find any two distinct messages m_1, m_2 with $H(m_1) = H(m_2)$. Attack scenario: An attacker prepares two documents (one benign, one malicious) that collide, obtains a signature on the benign one, then substitutes the malicious document. Because of the birthday paradox the expected work is only $\approx 2^{n/2}$.

Avalanche effect: A single-bit change in the input must change approximately half of the output bits in an unpredictable way. Without avalanche, differential cryptanalysis and related-key or near-collision attacks become practical.

2. Expected number of output bits that flip (ideal hash)

For an ideal n-bit hash that behaves like a random oracle, each output bit is independent and flips with probability 1/2 when any input bit is flipped. Therefore the expected Hamming distance between H(m) and $H(m \oplus e_i)$ is exactly n/2. For SHA-256 (n = 256) the expected number of changed bits is 128. Variance is n/4 = 64, so standard deviation ≈ 8 bits; observed averages after many trials concentrate tightly around 128.

```
E[ Hamming distance ] = n/2 = 128 bits for SHA-256
```

3. Why a toy hash (byte-sum / XOR-fold) fails avalanche and second pre-image

Consider a simple XOR-fold: $H(m) = m[0] \oplus m[1] \oplus \ldots \oplus m[L-1]$ (or a running byte sum modulo 256). Flipping one input bit flips at most one output bit (or a predictable linear combination). The avalanche is essentially zero. Mathematically the function is linear (or affine) over GF(2). Consequently, given m and H(m), an attacker can construct $m' = m \oplus \Delta$ where Δ is chosen so that the linear form still evaluates to the same value; second-pre-image search is trivial (linear algebra). Collision search is equally easy: any two messages whose bytes XOR to the same value collide.

4. Why a fixed-length digest must have collisions

By the pigeonhole principle: the set of all possible messages is infinite (or at least larger than 2^n), while the set of digests has cardinality 2^n. Therefore at least two distinct messages must map to the same digest. Collision resistance does not claim that collisions do not exist; it claims that finding them is computationally infeasible.

Student B – Experimental Validation

Design inputs of three lengths (e.g., 16 B, 64 B, 256 B). For each length perform ≥ 200 independent single-bit flips at random positions. Instrument the code to compute the Hamming distance between the original and mutated digests (both for the toy hash and for SHA-256). Record mean and standard deviation. Expected result: toy hash mean ≈ 1–2 bits; SHA-256 mean ≈ 128 ± 8. Any discrepancy larger than a few bits for SHA-256 indicates an implementation error (incorrect bit counting, endianness, or failure to re-hash after mutation).

Part B – Collision Effort and the Birthday Bound

Student A – Experimental Investigation

Truncate SHA-256 to the first k bits for $k \in \{16, 20, 24, 28, 32\}$. For each k run a classic birthday attack: generate random messages, store (digest → message) in a dictionary, stop at the first collision. Repeat each k at least 10–20 times and record the number of hashes computed. Plot observed effort versus $2^{k/2}$; the curve should track the theoretical line closely for small k and begin to deviate only when memory or constant factors dominate.

Student B – Birthday-Bound Reasoning

Let $N = 2^k$ be the size of the digest space. The probability that q random samples are collision-free is approximately $\exp(-q(q-1)/(2N))$. Setting this probability to ½ and solving yields $q \approx 1.177\sqrt{N} \approx 2^{k/2}$. Hence the expected effort to find any collision is $\Theta(2^{k/2})$, not 2^k.

Finding a collision with a *specific* given message is a second-pre-image problem and still costs $\approx 2^k$. The birthday attack only finds some pair; the attacker must be free to choose both messages in advance (classic chosen-prefix or identical-prefix collision setting).

Consequently a digest used for collision resistance must be at least 256 bits today (target 128-bit security). MD5 (128-bit) and SHA-1 (160-bit) were retired precisely because practical collision attacks fell well below the birthday bound (chosen-prefix collisions for MD5 in seconds, for SHA-1 in practice after 2017).

Q2 – Integrity Is Not the Same as Authentication: Hash, MAC and HMAC (7 Marks)

Student A – Attack and Construction Investigation

1. Plain hash is defeated when attacker controls both file and published digest
A software distributor that publishes only H(file) on the same page gives the attacker a free hand: replace file by file′ and simultaneously replace the published digest by H(file′). The verification procedure still succeeds. A plain hash therefore provides integrity only against accidental corruption, never against a malicious party that can modify the reference value.

2. Length-extension attack on naïve MAC = H(key ■ message)
Any Merkle–Damgård hash (MD5, SHA-1, SHA-256, …) exposes its entire internal chaining state in the final digest. Given H(key ■ m) and the length of key ■ m, an attacker can reconstruct the padding that the original computation applied, then continue the compression function with arbitrary extra blocks of his choice. The resulting value is exactly H(key ■ m ■ padding ■ extra). The attacker never needs to know the key; he only needs the digest and the length of the secret prefix.

Property that enables the attack: the final output of a Merkle–Damgård hash is the full chaining value after the last block. There is no finalisation that hides state.

Student B – Comparative Security Analysis

HMAC structure: HMAC(K, m) = H((K ⊕ opad) ■ H((K ⊕ ipad) ■ m)). Two nested hashes with distinct derived keys. Because the outer hash is applied to a fixed-length string (the inner digest), length-extension of the outer hash yields a value that is not a valid HMAC under the original key. The construction is therefore immune to the classic length-extension attack even when the underlying hash is Merkle–Damgård.

Mechanism	Integrity	Authentication	Notes
Hash only (public digest)	Yes (accidental)	No	Attacker can change both file & digest
Naïve H(key■msg)	Yes	Weak	Vulnerable to length extension
HMAC	Yes	Yes	Resists length extension; key must stay secret

Why a MAC cannot provide non-repudiation: Both parties that share the secret key can produce a valid MAC. Consequently a recipient cannot prove to a third party that the sender (and only the sender) generated the tag; the recipient could have forged it himself. Non-repudiation requires an asymmetric primitive (digital signature) whose private key is known only to the signer.

Joint Design Question

Download verification: Prefer a digital signature (or a hash of the file that is itself signed / published via a trusted channel such as a certificate-pinned HTTPS endpoint). A plain SHA-256 digest on the same web page is insufficient. Residual risk: compromise of the signing key or of the distribution channel that delivers the public key / certificate.

Internal message protection: Use HMAC (or an AEAD construction). The shared key must be kept secret and rotated. Residual risk: key compromise, implementation bugs (e.g., timing side-channels), or failure to authenticate associated data.

Q3 – Digital Signatures: Hash-then-Sign and What Can Break Them (7 Marks)

Student A – Algorithmic Derivation

Why hash before signing: Signing a raw arbitrary-length message would require either (a) a signature scheme that accepts long inputs (rare and slow) or (b) breaking the message into blocks and signing each, which opens existential forgery via reordering or splicing. Hashing first reduces the problem to signing a short fixed-length digest, binds the entire message, and inherits the collision resistance of the hash. Without hashing, textbook RSA is multiplicatively malleable: $\sigma(m_1)\cdot\sigma(m_2) \equiv \sigma(m_1\cdot m_2) \pmod n$.

RSA verification correctness: Signature generation: $s \equiv H(m)^d \pmod n$. Verification: compute $s^e \bmod n$ and test equality with $H(m)$. By Euler's theorem / RSA correctness, $(H(m)^d)^e \equiv H(m)^{de} \equiv H(m)^{1 + k\phi(n)} \equiv H(m) \pmod n$ when $\gcd(H(m), n) = 1$ (which holds with overwhelming probability for a random digest).

DSA / ECDSA verification (sketch): Signature (r, s) on hash $z = H(m)$. Verifier computes $w = s^{-1} \bmod q$, $u_1 = z\cdot w \bmod q$, $u_2 = r\cdot w \bmod q$, then $V = (u_1\cdot G + u_2\cdot Q)$ and checks that the x-coordinate of V equals r. Algebraically this recovers the same point $k\cdot G$ that the signer used, because the discrete-log relations cancel.

Security goals contributed: hash provides integrity of the message; private-key possession provides authentication of the signer; the combination of a public-key signature and a trustworthy public-key binding (certificate) yields non-repudiation under the usual assumptions.

Student B – Attack Experiment and Evidence

Nonce reuse in DSA/ECDSA: Two signatures (r, s_1), (r, s_2) that share the same k produce the identical r. Then $k = (z_1 - z_2)\cdot(s_1 - s_2)^{-1} \bmod q$ and the private key $d = (s_1\cdot k - z_1)\cdot r^{-1} \bmod q$. This is pure algebra; no discrete-log hardness is needed once k is reused. Historic examples: Sony PS3 firmware signing, early Android Bitcoin wallets.

Textbook RSA forgery (multiplicative): Given $\sigma_1 = m_1^d$ and $\sigma_2 = m_2^d$, the product $\sigma_1\cdot\sigma_2$ is a valid signature on $m_1\cdot m_2$. Hashing before signing destroys the algebraic structure and prevents the attack.

These experiments illustrate "correct mathematics, unsafe implementation": the number-theoretic relations hold, yet the protocol becomes completely insecure when randomness is mishandled or the hash is omitted.

Joint Conclusion – Non-repudiation

A valid signature proves that someone who possessed the private key authorised the message. True non-repudiation additionally requires: (1) the private key was under the exclusive control of the claimed signer at the time of signing, (2) a reliable timestamp or notary that proves when the signature was created, and (3) a trustworthy public-key infrastructure that binds the public key to the signer's identity. The signer can attempt to deny by claiming key compromise, stolen credentials, or a flawed implementation that leaked the key. A trusted third-party timestamping authority and hardware security modules raise the bar for such claims.

Q4 – Trust Through Certificates: X.509 Chain Validation (7 Marks)

Student B – Trust Model & Validation Logic

Validation algorithm (pseudocode summary):

1. Build the candidate chain from end-entity up to a trusted root.
2. For each certificate in the chain (starting from the leaf): verify the signature using the issuer's public key; check that issuer name matches subject of the next certificate; check notBefore / notAfter; verify basicConstraints (CA flag must be true for intermediate certificates); check keyUsage / extendedKeyUsage; perform name matching (CN / SAN) against the expected identity; consult revocation status (CRL or OCSP).
3. The final certificate must be a trust-anchor already present in the client's trust store.

Each step defeats a concrete attack: signature check prevents forgery; validity window prevents use of expired certificates; basicConstraints stops an end-entity certificate from acting as a CA; name matching prevents hostname impersonation; revocation check limits the window after a private-key compromise.

Trust in the root cannot be verified cryptographically inside the system; it is a pre-configured out-of-band decision (the root is shipped with the OS / browser or configured by an administrator). X.509 binds identity to a public key by a signature from a higher authority; that binding is what makes a subsequent signature on data meaningful—the verifier can follow the chain back to a root it already trusts.

Student A – Failure-Case Experiments

Failure Case	Rejected by
Expired certificate	Validity period check
Not-yet-valid certificate	Validity period check
Wrong issuer / broken chain	Issuer–subject matching or signature check
Tampered certificate field	Signature verification fails
Hostname / SAN mismatch	Name matching step
End-entity used as CA	basicConstraints CA flag
Revoked certificate	Revocation check (CRL/OCSP)

CRL vs OCSP: CRL is a periodically published signed list (larger, cacheable, offline-friendly, no per-query privacy leak). OCSP answers a single serial-number query (small, fresher, but reveals the certificate being checked to the responder and requires network reachability). A "fail-open" policy that accepts a certificate when the revocation source is unreachable completely negates the security of revocation; a fail-closed (or soft-fail with short cache) policy is required for high-assurance environments.

Joint Engineering Decision

Damage after CA compromise or mis-issuance is limited by: short certificate lifetimes, rapid revocation (OCSP stapling + short CRL nextUpdate), certificate transparency logs, and name constraints / path-length constraints. To shorten the window: reduce maximum validity (e.g., 90 days or less), mandate OCSP stapling, deploy automated monitoring of CT logs, and keep offline root keys with strict dual-control ceremonies.

Q5 – Kerberos: Authentication Without Sending Passwords (6 Marks)

Student A – Protocol Design

Message-sequence (simplified):

1. Client → AS: identity, TGS name, nonce.
2. AS → Client: $\{TGT\}_{K_{TGS}}$, $\{K_{c,TGS}, \text{nonce}, \ldots\}_{K_c}$ (K_c derived from password).
3. Client → TGS: TGT, $\text{Authenticator}=\{c, \text{timestamp}\}_{K_{c,TGS}}$, service name.
4. TGS → Client: $\{\text{ServiceTicket}\}_{K_{service}}$, $\{K_{c,s}, \ldots\}_{K_{c,TGS}}$.
5. Client → Service: ServiceTicket, $\text{Authenticator}=\{c, \text{timestamp}\}_{K_{c,s}}$.
6. Service → Client (optional mutual auth): $\{\text{timestamp}\}_{K_{c,s}}$.

The Ticket-Granting Ticket (TGT) lets the client obtain service tickets without re-entering the password; the long-term key K_c is used only once per login session. The authenticator proves freshness and possession of the session key; the timestamp inside it, together with a limited lifetime on the ticket, prevents replay. Mutual authentication is obtained when the service returns a message encrypted under the new session key that contains a recognisable value derived from the client's authenticator.

Student B – Attack and Failure Evidence

Replay of a captured authenticator fails when the service remembers recent timestamps (or sequence numbers) inside the acceptance window; without that cache the replay succeeds. An expired ticket is rejected by the lifetime check; excessive clock skew is rejected by the timestamp window. If the KDC is unavailable, no new TGTs or service tickets can be issued (cached tickets may still work until they expire). A stolen service ticket allows impersonation of the client to that service until the ticket expires; an offline dictionary attack on the AS reply can recover a weak password.

Cross-Understanding

The client never sends its password: the password only decrypts the AS reply that contains the TGT session key. The service learns the authenticated client identity that the KDC placed inside the service ticket; it does not learn the password or any long-term key of the client.

Q6 – Advanced Security Mini-Investigations (7 Marks)

Student A – Hash Weakness and Signature Forgery

Collision attack finds any two messages that hash to the same value; second-pre-image attack finds a different message that hashes to the same value as a *given* message. Using the truncated-hash collision finder of Q1, produce $m_1 \neq m_2$ with $H_k(m_1) = H_k(m_2)$. A signature on $H_k(m_1)$ is then also a valid signature on m_2. The attacker must prepare both messages before obtaining the signature (chosen-prefix setting). Real-world MD5/SHA-1 collisions enabled forged X.509 certificates (rapid-ssl / Flame). A modern CA must refuse MD5 and SHA-1; SHA-256 or stronger is required. Recommended policy: SHA-256 or SHA-384/512 for all signatures; prefer SHA-3 or BLAKE2 where available; never sign a digest shorter than 256 bits.

Student B – Kerberos versus X.509 and Non-Repudiation

Aspect	Kerberos	X.509 / PKI
Key type	Symmetric (shared)	Asymmetric (public/private)
Trusted third party	KDC (online)	CA (mostly offline)
Online dependence	High (KDC needed for tickets)	Low after certificate issuance
Scalability	Good inside realm; cross-realm harder	Excellent (hierarchical)
Revocation	Short ticket lifetimes	CRL / OCSP
Typical environment	Enterprise SSO, campus	Internet, code signing, legal docs
Non-repudiation	No (shared secret)	Yes (private key exclusive)

Kerberos cannot give non-repudiation because the service and the KDC also know the session keys; any of them could have produced the authenticators. X.509 signatures can, provided the private key remains under the sole control of the signer. Single point of failure for Kerberos is the KDC; for X.509 it is the root CA (or any intermediate that is overly trusted). Number of keys: pairwise shared = $n(n–1)/2$; Kerberos $\approx n + s$ (users + services); PKI $\approx n$ (each party has one key pair) plus a few CA keys.

Recommendation: (i) campus SSO $\rightarrow$ Kerberos (or a modern SSO protocol built on similar ideas); (ii) legally binding electronic contracts $\rightarrow$ X.509 / qualified electronic signatures.

Comparison of Signature Schemes (required evidence)

Scheme	Key material	Nonce / randomness	Forgery note
RSA (hash-then-sign)	(n,e)/(n,d)	None (deterministic if pure)	Malleable without hash/padding
DSA / DSS	p,q,g,x / y	Per-signature k mandatory	Nonce reuse $\rightarrow$ full key recovery
ECDSA	Curve, G, d / Q	Per-signature k mandatory	Same as DSA
ElGamal signature	p,g,x / y	Fresh k each signature	Poor k handling leaks key
Schnorr	p,q,g,x / y	Commitment randomness	Predictable challenge / nonce reuse fatal

Schnorr (toy): Commitment $R = g^k$, challenge $c = H(R \blacksquare m)$, response $s = k + c \cdot x \bmod q$. Verification: $g^s = R \cdot y^c$. The challenge must be unpredictable (random oracle / honest hash) otherwise the prover can cheat; nonce (k) reuse again permits algebraic recovery of x.

Q7 – Design a Secure Authentication and Integrity Scheme from First Principles (10 Marks)

Problem Context Recap

Firmware updates for infusion pumps must guarantee: (a) integrity, (b) authenticity of origin (manufacturer), (c) non-repudiation. Pumps have limited memory, are often offline, and possess unreliable clocks. Some pumps also need to authenticate to a hospital server.

Stage 1 – Independent Design Ideas (retained evidence)

Student A sketch: Manufacturer holds an offline root and an online intermediate signing key. Each firmware image is hashed with SHA-256 and signed with the intermediate key (RSA-3072 or ECDSA P-256). The signature and a short certificate chain travel with the image. Version number and a monotonic counter are included inside the signed payload to prevent rollback. Pumps store the root public key in immutable memory (ROM / eFuse). Offline verification needs only the chain and the image; no network and no accurate clock are required beyond a coarse “not-before” that can be ignored or replaced by a version check.

Student B sketch: Similar signature approach, plus an optional hospital-side Kerberos or certificate-based mutual authentication for the service session after the update. Emphasis on revocation via a short-lived intermediate and a published CRL that pumps can obtain when they occasionally come online.

Stage 2 – Comparison & Stage 3 – Joint Final Design

Agreement: asymmetric signature by the manufacturer is mandatory for non-repudiation and offline verification. Differences: degree of reliance on clocks and online revocation. Retained ideas: hash-then-sign with SHA-256, short certificate chain, monotonic version / anti-rollback counter, root public key in immutable storage, optional secondary authentication for hospital service sessions.

Threat model: Network attacker who can modify or replay update packages; malicious hospital insider; compromised manufacturer signing key; physical extraction attempts on a single pump.

Security goals & mechanisms:

• Integrity → SHA-256 digest inside the signed structure.
• Authentication of origin → digital signature under manufacturer intermediate key + X.509 chain to a trust anchor burned into the pump.
• Non-repudiation → private key known only to manufacturer; signature retained with the image.
• Anti-rollback → signed monotonic version / sequence number; pump rejects any version ≤ current.
• Limited revocation → intermediate certificates with short lifetime (e.g., 1 year) + CRL/OCSP when connectivity exists; fail-closed for critical updates if possible.

Key & certificate management: Offline root CA (air-gapped, dual control). Intermediate signing CA issued by root, validity 1–2 years, keyUsage = digitalSignature + codeSigning. Rotation: new intermediate issued before old expires; pumps accept both during a transition window. Revocation: publish CRL; pumps that can phone home check it; otherwise rely on short lifetime.

Message flow (update):

1. Manufacturer builds firmware image F, computes h = SHA-256(F ■ version ■ meta).
2. Signs h with intermediate private key → σ.
3. Packages (F, version, meta, σ, certificate chain).
4. Distribution via any channel (USB, hospital network, OTA).
5. Pump verifies chain to embedded root, checks version > current, verifies σ, then installs.

Offline / wrong-clock behaviour: Verification never requires a real-time clock; only the monotonic version counter and the cryptographic checks are mandatory. A coarse not-before date may be ignored or replaced by the version policy when the clock is known to be unreliable.

Pseudocode (verification):

```
function VerifyUpdate(pkg, root_pubkey, current_version):
  chain ← pkg.certificates
  if not ValidateChain(chain, root_pubkey): return REJECT
  if pkg.version ≤ current_version: return REJECT // anti-rollback
  h ← SHA256(pkg.firmware ■ pkg.version ■ pkg.meta)
  if not VerifySignature(chain[0].pubkey, h, pkg.σ): return REJECT
  // optional: if online, check revocation status of chain[0]
```

```
return ACCEPT
```

Stage 4 – Validation Responsibilities

Student A (security argument): Collision on SHA-256 costs ≈ 2^{128} work (birthday). Forging an RSA-3072 / ECDSA-P256 signature without the private key is infeasible under standard assumptions. Compromise of the intermediate key is limited by its lifetime and by the anti-rollback counter (old signed images cannot be re-introduced). Residual risks: physical extraction of a pump's root key (mitigated by eFuse / secure element), supply-chain compromise before the root is burned, and implementation bugs.

Student B (experimental): Test vectors – valid package (accept); modified firmware (reject); modified signature (reject); older version (reject); expired intermediate (reject if validity checked); wrong root (reject). Compare against a baseline that only publishes a bare SHA-256 digest: the baseline fails every authenticity and anti-rollback test. Record verification cost (one SHA-256 + one public-key operation + chain walk) and metadata size (signature + 2–3 certificates ≈ a few kilobytes).

Stage 5 & 6 – Cross-Defence and Joint Evaluation

Integrity is guaranteed by the hash; authentication and non-repudiation by the signature under a private key bound by the X.509 chain. Costs: modest storage for the root key and a few certificates, one signature verification per update (acceptable even on constrained MCUs). Failure modes still possible: manufacturer key compromise before revocation propagates, flawed RNG or side-channel in the signing HSM, or a bug that skips the version check. A pure HMAC design would be lighter but cannot provide non-repudiation and requires a shared secret distributed to every pump (key-management nightmare). Kerberos-style tickets are appropriate for the hospital service-session authentication after the pump is updated, not for the firmware image itself.

End of Solutions Document

All theoretical derivations, attack explanations, comparison tables and design decisions required by the assignment are provided. Experimental sections give concrete measurement plans and expected numerical results so that implementation and reporting can proceed directly.